The Ch(e)at Code

Agent Patterns for Copilot Chat

↑ ↑ ↓ ↓ ← → ← → B A START

ISSUE #007 • MARCH 31, 2026 • WEEKLY

Your customer just asked: "Can we build one agent that answers questions across all our projects — even when those projects share nothing in common?" They have design docs in SharePoint, decision records in repos, and meeting notes from Issue #006's pipeline. Each data source is a silo. No single agent connects them today.

This issue's pattern: A cross-project knowledge agent built in Copilot Studio that uses enriched Azure AI Search to synthesize answers across project boundaries — even when the source documents never reference each other.

🎮 AGENT SPOTLIGHT

The Cross-Project Knowledge Agent

Pattern by Tyson Dowd • Architectured from 1:1 discussion (March 24, 2026)

The scenario: An enterprise runs 6–8 concurrent projects across engineering, compliance, and product teams. Each team stores artifacts in its own SharePoint site. A new hire joins and asks: "What decisions from Project Alpha's auth design affect Project Beta's compliance timeline?" Today, the answer is "go ask three people and hope they remember." Nobody — and no agent — can connect those dots.

The solution: A Copilot Studio agent backed by a single Azure AI Search index that ingests documents from all project sources. The secret: an enrichment pipeline that stamps every document with *what other documents it relates to* — so the agent can traverse connections that were never explicitly made by any human.

Why Copilot Studio: The conversational layer is simple — generative answers pointed at Azure AI Search. The intelligence lives in the *index enrichment*, not the agent logic. This means you build the hard part

once (the enrichment pipeline) and get a working agent on Day 1 with zero custom code in Copilot Studio itself.

🧩 **PATTERN BREAKDOWN**

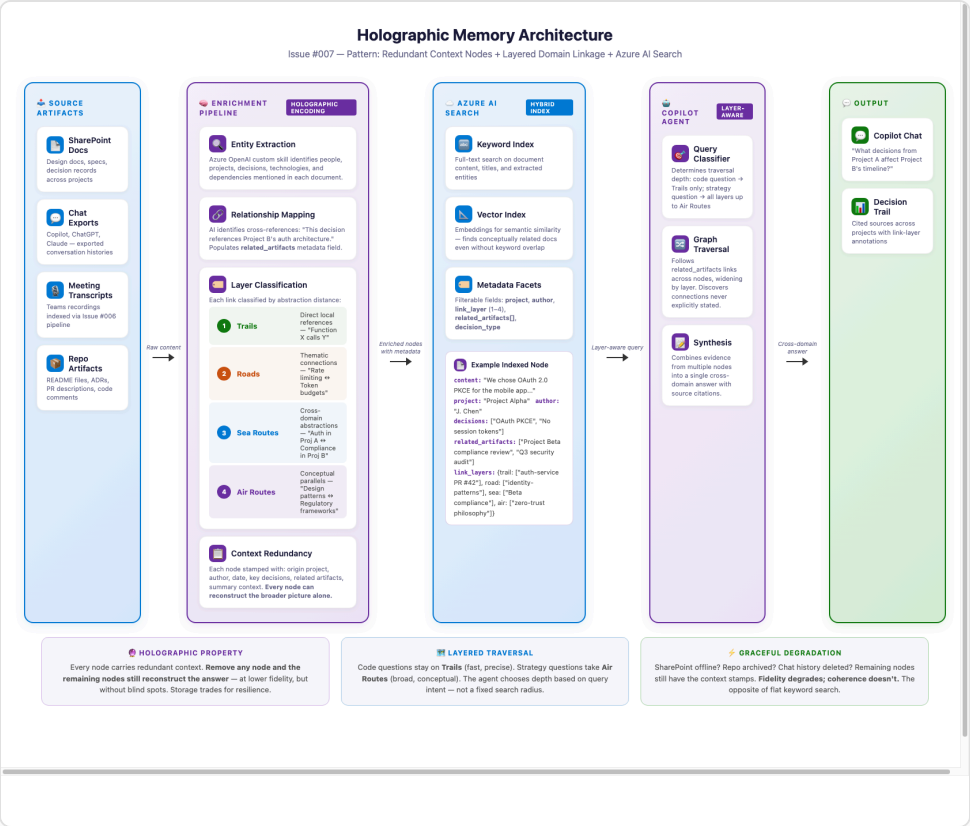
Enriched Knowledge + Generative Answers

Copilot Studio + Azure AI Search (custom skillset) + Power Automate + Azure OpenAI

Why this matters: Copilot Studio's built-in SharePoint knowledge source searches *within* a site. If your customer's knowledge spans five SharePoint sites, three repos, and a meeting archive, the agent can't synthesize across them. Azure AI Search solves the retrieval problem — but only if the index carries enough context to connect documents that don't reference each other. That's what the enrichment pattern adds.

END-TO-END BUILD

```
SharePoint sites + repos + meeting transcripts
  ↓
Power Automate → triggered on new/modified documents
  ↓
Azure OpenAI custom skill → extracts entities, decisions, relationships
  ↓
Stamps each doc with related_artifacts metadata field
  ↓
Azure AI Search hybrid index (vector + keyword + metadata facets)
  ↓
Copilot Studio agent → generative answers + custom system prompt
  ↓
Cross-project answer with source citations
```



Full architecture: Sources → Power Automate enrichment → Azure AI Search → Copilot Studio agent — click to enlarge

Four components, in build order:

1. **Azure AI Search index with custom skillset.** Create a search index with these fields beyond the standard content: `project_name`, `author`, `decisions[]`, `related_artifacts[]`, `summary_context`. Add a custom skill that calls Azure OpenAI to populate these fields during indexing. The `related_artifacts` field is the key — it lists other documents this one connects to, even across projects. Make `project_name` and `decisions` filterable, not just searchable.

2. **Power Automate ingestion flow.** Trigger: "When a file is created or modified" in each project's SharePoint site. Actions: (a) extract document text, (b) call Azure OpenAI with a prompt like *"Extract the project name, key decisions, people mentioned, and any references to other projects or documents"*, (c) push the enriched document into Azure AI Search via the Search REST API. This runs automatically as new content arrives — no manual indexing.

3. **Copilot Studio agent with generative answers.** Create the agent, add Azure AI Search as a knowledge source, and configure generative answers. The critical piece: the **custom system prompt** that instructs the agent to synthesize across projects. Use something like: *"When answering, search across all projects. If you find related decisions in different projects, explicitly connect them and cite both sources. Always state which projects*

your answer draws from." This is what turns a basic search agent into a cross-project intelligence tool.

4. **Plugin action: "Index this conversation."** Build a Power Automate cloud flow exposed as a plugin action. When a user pastes an exported chat history (from Copilot, Claude, ChatGPT), the flow parses it, extracts decisions and references, and indexes it into the same Azure AI Search instance. This turns ephemeral conversations into durable, searchable knowledge nodes. Users actively feed the system.

Key insight: The agent itself is trivially simple — it's generative answers pointed at a search index. **All the intelligence lives in the enrichment pipeline.** This means you can demo the agent in Day 1 with a manually enriched index, then automate the enrichment in Day 2 with Power Automate. The customer sees value immediately.

Why "redundant context" matters: Each indexed document carries `summary_context` — a brief description of the project, the team, and the domain. This means if the agent retrieves a single document, it already knows enough context to connect it to a question about a different project. Like a hologram: each piece contains a low-resolution image of the whole. Lose a SharePoint site? The remaining documents still carry enough context to answer — at lower fidelity, but without blind spots.

⚠ **What Copilot Studio can't do (yet):** The agent can't follow `related_artifacts` links hop-by-hop like a graph database. It retrieves the top-N search results and synthesizes from those. For deeper traversal, add a pre-query Power Automate flow that expands related documents before passing them to generative answers. Also: the `search_filter` parameter in the Azure AI Search connector lets you scope queries by project or metadata — use this to build topic-specific variants of the same agent.

🛠️ **TRY THIS NOW**

Build the Demo in One Day

1. **Pick two SharePoint sites from two different projects.** Choose ones with design docs, decision records, or architecture docs. You need 10–15 documents per site — enough to have cross-project overlap in people, technologies, or decisions. Don't

over-engineer the selection — any two real projects will have implicit connections.

2. **Manually enrich 20 documents for the demo.** For each doc, add a JSON metadata file with: `project_name`, `decisions`, `related_artifacts`, `summary_context`. Yes, this is manual for the demo — the Power Automate pipeline automates it in production. It takes about 2 minutes per doc with Copilot Chat helping you write the metadata.

3. **Create the Azure AI Search index and the Copilot Studio agent.** Index the 20 enriched docs. Create a Copilot Studio agent, add Azure AI Search as a knowledge source, enable generative answers, and paste in the cross-project system prompt. Test with: *"What decisions from Project Alpha affect Project Beta?"*

4. **Demo the "impossible" question.** Ask a question that no single document answers — one that requires connecting a decision in Project A with a constraint in Project B. If the agent synthesizes an answer citing both projects, you've demonstrated the pattern. If not, the `related_artifacts` metadata needs richer cross-references.

💡 **The line that opens doors:** "What if a new hire could ask one question and get an answer that connects decisions across every project your team has ever run — with sources cited from documents that were never explicitly linked?"

🏆 **WHERE THIS PATTERN LANDS**

- **Professional services & consulting** — "What lessons from the Acme engagement apply to our new Baker project?" Cross-engagement knowledge reuse without senior partner availability
- **Engineering & R&D** — "Why did Team A choose OAuth PKCE and does that affect Team B's compliance approach?" Architecture decision reuse across squads
- **Regulated industries (pharma, finance)** — "Show me all decisions across all projects that affect our Q4 audit scope." Compliance traceability without manual cross-referencing

- **M&A & integration** — rapidly connecting knowledge bases from two merging organizations where nothing was linked before
- **New hire onboarding** — "Catch me up on everything that happened on this project before I joined." The agent synthesizes from meeting notes, design docs, and decision records simultaneously

Positioning tip: This is "AI maturity, not AI novelty." Your customer has chatbots. They've done RAG. This is the next step: *relational knowledge* — an agent that doesn't just find documents, but finds **connections between documents** that no human explicitly created. Lead with the "impossible question" demo — it's the moment the room goes quiet.

⚡ **QUICK TIPS**

✓ **Use Copilot Chat to generate the metadata.** For the manual enrichment step, paste each document into Copilot Chat and ask: *"Extract the project name, key decisions, people mentioned, and any references to other projects or documents. Return as JSON."* It takes 2 minutes per doc instead of 15.

✓ **The enrichment prompt matters more than the agent prompt.** Spend your tuning time on the Azure OpenAI prompt used in the custom skill, not on the Copilot Studio system message. If the metadata is rich, the agent's generative answers are automatically better.

✓ **Combine with Issue #002 (Scoped Multi-Source Search)** for the retrieval pattern and **Issue #006 (Meeting-to-Knowledge Pipeline)** for the meeting transcript source. This agent is the downstream consumer of both patterns.

✓ **Budget for 3–5x index storage.** The redundant context metadata (summary_context, related_artifacts) increases index size significantly. This is a deliberate trade-off: you're spending storage to buy cross-project recall. Size your Azure AI Search tier accordingly.

Got an agent or pattern to share?

Reply to this email and you might be featured in a future issue.

The Ch(e)at Code

AIWF Team • AI Business Solutions

Questions or ideas for the next issue? Reply to this email.

Microsoft Internal • Not for external distribution